SOFTWARE QUALITY ASSURANCE · CAPSTONE LAB

API Quality Assurance for Real-World Global Systems

Student handout · Dr. NEGM · 5-tiered exercises · 100 marks · +10 bonus

1. Mission & Context

This capstone brings together every skill from Labs 1 through 9 — test planning, Selenium, TestNG, Postman, REST-Assured — and points them at the APIs that quietly run the modern world.

You will write one Maven project with five TestNG classes. Each level mirrors a real company's nightly regression pattern:

Level	Domain	Real-world mirror	Marks
L1 · Easy	E-Commerce	Amazon · Shopify · Jumia · Noon	**10**
L2 · Beginner+	Service Portals	ServiceNow · Salesforce · Gov.uk · Digital Egypt	**15**
L3 · Intermediate	Mobile Banking	Revolut · HSBC · Chase · CIB (PSD2)	**15**
L4 · Advanced	Mobile Payments	PayPal · Apple Pay · M-Pesa · InstaPay/Fawry	**20**
L5 · Advanced	Mixed Domains	Shopify / Stripe nightly regression suite	**25**

2. Project Setup

You've been given a starter Maven project. From the project root:

```
mvn -q -DskipTests compile    # compile everything
mvn -Dtest=L1_GetCatalogTest test   # run just level 1
mvn clean test                # run the full 5-level suite
```

Required on your machine: JDK 17+, Maven 3.8+.

Dependencies are pre-wired in pom.xml: REST-Assured 5.4.0, TestNG 7.10.2, json-simple 1.1.1, hamcrest 2.2 and json-schema-validator.

3. Level 1 - Easy: Fetch the Catalog

Real-world context · E-commerce

When you open the Amazon app, the very first HTTP call is GET /catalog/products. Shopify, Jumia, and Noon do the same. Before any Buy button works, the front-end must trust that the catalog endpoint returns 200 OK with a valid JSON body. This is the test QA engineers run thousands of times a day.

Your task

- Send a GET request to https://fakestoreapi.com/products
- Assert that the HTTP status code is 200
- Assert that the response content type is JSON
- Assert that the first product's id equals 1
- Print the full response body using .log().all()

Hints

- Use given().when().get(URL).then() — the classic BDD chain.
- Static-import RestAssured.given and Matchers.equalTo.
- JSON path for the first element's id is "[0].id".

4. Level 2 - Beginner+: File a Service Ticket

Real-world context · Service portals

ServiceNow, Salesforce Service Cloud, Egypt's Digital Gateway, and the UK's Gov.uk Notify all expose a POST /tickets endpoint. Every citizen complaint travels through this exact API. QA must guarantee that valid submissions get a 201 Created and a real ticket ID — otherwise the public loses trust in their government.

Your task

- POST to https://jsonplaceholder.typicode.com/posts with the following JSON body:

```
{ "title": "Power outage in Zone 4",
  "body":  "No electricity since 18:00.",
  "userId": 7 }
```

- Assert status code 201.
- Assert the returned title equals the title you sent.
- Assert the returned id field is not null (server generates it).

Hints

- Build the body with org.json.simple.JSONObject.
- Always set .contentType("application/json").
- POST → 201, not 200. Don't confuse this with GET.
- ServiceNow's real ticket field is called sys_id. Field names matter.

5. Level 3 - Intermediate: Update Bank Customer

Real-world context · Mobile banking (PSD2 / Open Banking)

Revolut, Monzo, Chase, HSBC and CIB all expose a PUT /customers/{id} endpoint to update profile data — phone, address, KYC fields. Under PSD2 (EU) and Open Banking (UK), banks are legally obligated to keep these endpoints reliable. A failed PUT is a regulatory incident, not just a bug.

Your task

- PUT to https://reqres.in/api/users/2 with body { "first_name":"Khaled", "job":"VIP Customer" }.
- Assert status code 200.
- Assert Content-Type contains "application/json".
- Assert updatedAt is not null.
- Assert response time < 3 seconds (.time(lessThan(3000L))).

Hints

- Use containsString rather than equalTo for content type — servers may append charset.
- Call .extract().response() to reuse values downstream.
- PSD2 mandates 99.5% monthly availability; your SLA assertion is the early-warning signal.

6. Level 4 - Advanced: Authenticated P2P Payment

Real-world context · Mobile payments

PayPal, Apple Pay, Venmo, Kenya's M-Pesa, Egypt's InstaPay and Fawry all run the same two-step dance: (1) authenticate to get a bearer token, (2) call the send-money endpoint with that token in the Authorization header. If your tests can't survive a token round-trip, they cannot test any modern payment system.

Your task

Step 1 — authenticate

- POST https://reqres.in/api/login with body { email, password }.
- Expect status 200 and a non-null token. Store it in a static class field.

Step 2 — send a payment

- POST https://reqres.in/api/transactions with header Authorization: Bearer <token>.
- Body: { "to":"+201001234567", "amount":250, "currency":"EGP" }.
- Accept 200 OR 201 (use anyOf(is(200), is(201))).
- Assert body id is not null and response time < 5 seconds.

Hints

- Use @Test(priority = 1) and @Test(priority = 2) to force the order.
- Real PSPs use OAuth 2.0 with refresh tokens — same pattern, just longer-lived.
- In production, read credentials from System.getenv() — never hard-code.

7. Level 5 — Advanced level: End-to-End Checkout & Refund

Real-world context · Mixed domains

This is the shape of the nightly regression suite at Shopify, Noon, Jumia, and Amazon — a single Java class that touches catalog, customer, checkout, payment, and refund APIs in sequence. If any link breaks, deployment is blocked. You're now writing the same test that holds the door open for production.

Your mission

One TestNG class. Five @Test methods, each with a priority. Five endpoints chained by IDs:

- 1. registerCustomer → reqres /register (capture userId + token)
- 2. browseCatalog → FakeStore /products/1 (capture price)
- 3. addToCart → FakeStore /carts (capture cartId)
- 4. payForCart → reqres /transactions (use token, capture txnId)
- 5. refundTxn → DELETE reqres /transactions/{txnId}

Acceptance criteria

- Every step assert BOTH status code AND at least one body field.
- Every call completes under 3 seconds (SLA).
- IDs pass cleanly between methods via static class fields.
- A @BeforeClass method sets RestAssured.useRelaxedHTTPSValidation().
- (Bonus) produce an Allure HTML report for the whole suite.

8. Grading & Submission

Criterion	Weight	Excellent (full marks)
Level 1 GET catalog passes	**10%**	All 4 assertions, log().all() shown.
Level 2 POST ticket passes	**15%**	Body round-tripped, id verified.
Level 3 PUT bank update passes	**15%**	Status + schema + SLA timing.
Level 4 Auth + payment passes	**20%**	Token chained, headers correct.
Level 5 End-to-end suite passes	**25%**	All 5 steps green, IDs chained.
Code quality & style	**10%**	DRY, base class, readable names.
Real-world reflection (1 page)	**5%**	Maps levels to real companies.
TOTAL	**100**	**+10 bonus (max grade capped at 100)**

Deliverables

- A zip of your Maven project (without the target/ folder).
- A screenshot of the green TestNG runs across all five levels.
- A one-page reflection mapping each level to a real company.
- (Bonus) Allure HTML report folder.

File name

`SQA_ Capstone Lab_<StudentID>_<Name>.zip`

Deadline

- Next April 26th, 2026, 0900 Cairo local time, via the LMS.
- Late penalty: −10 marks/day, capped at −50.
- Plagiarized tests earn an automatic 0 plus a course report.

A Note

- Pair discussion is encouraged. Final code must be your own.